Experiment No. 7

Title

PHP Form Handling with MySQL CRUD Application

Aim

To develop a dynamic web application using PHP and MySQL that performs **Create, Read, Update, and Delete (CRUD)** operations on a database.

Software Required

• Text Editor (VS Code / Notepad++)
• Web Browser (Chrome / Firefox)
• XAMPP / WAMP Server
• PHP
• MySQL Database

Theory

PHP (Hypertext Preprocessor) is a server-side scripting language widely used for developing dynamic web applications. PHP scripts are executed on the server and the output is sent to the browser as HTML.

MySQL is a relational database management system used to store structured data in tables consisting of rows and columns. It allows applications to perform operations such as inserting, retrieving, updating, and deleting data.

When PHP is combined with MySQL, developers can build dynamic web applications that interact with databases to store and manage user information.

In this experiment, a simple **student management system** is implemented where user data is stored in a database and managed through web forms.

The application demonstrates **CRUD operations**, which are the fundamental operations performed on database records.

Operation	Description
Create	Insert new records into the database

Read	Retrieve and display stored records
Update	Modify existing records
Delete	Remove records from the database

The system works as follows:

1. The user enters information into an HTML form.
2. The form data is sent to a PHP script using the **POST method**.
3. PHP processes the request and interacts with the MySQL database.
4. The database performs the required CRUD operation.
5. The updated results are displayed on the webpage.

This demonstrates how server-side scripting and database integration enable dynamic web applications.

PHP Database Connection Syntax

```
$conn = mysqli_connect("localhost","root","","college_db");
```

• **mysqli_connect()** establishes a connection to the MySQL database.
• **localhost** refers to the local server.
• **root** is the default MySQL username.
• **college_db** is the database name.

MySQL CRUD Query Syntax

Insert (Create)

```
INSERT INTO students(name,email) VALUES('John','john@gmail.com');
```

Select (Read)

```
SELECT * FROM students;
```

Update

```
UPDATE students SET name='Rahul' WHERE id=1;
```

Delete

```
DELETE FROM students WHERE id=1;
```

Procedure

1. Install and start **XAMPP/WAMP server**.
2. Start **Apache** and **MySQL** services.

3. Open **phpMyAdmin** in a web browser.
4. Create a new database named **college_db**.
5. Create a table named **students**.
6. Write a PHP script to establish database connection.
7. Design an HTML form to collect student information.
8. Implement PHP scripts to insert, update, delete, and display records.
9. Execute the application in the browser.
10. Verify that CRUD operations work successfully.

SQL Query

```
-- Create the database
CREATE DATABASE IF NOT EXISTS college_db;
-- Select the database
USE college_db;
 -- Create the students table
CREATE TABLE IF NOT EXISTS students ( id INT AUTO_INCREMENT PRIMARY KEY, name
VARCHAR(100) NOT NULL, email VARCHAR(100) NOT NULL, created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP );
```

Database Setup (db.php)

Ensure your MySQL server is running and you've created the college_db database.

```
<?php
$host = "localhost";
$user = "root";
$pass = "";
$dbname = "college_db";

$conn = new mysqli($host, $user, $pass, $dbname);

if ($conn->connect_error) {
   die("Connection failed: " . $conn->connect_error);
}
?>
```

The Interface (index.php)

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Student Management System</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <style>
        body { background: #f8fafc; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; }
        .container-card { background: white; padding: 30px; border-radius: 12px; box-shadow: 0 10px 25px
rgba(0,0,0,0.05); }
        .table th { background: #2563eb; color: white; border: none; }
        .btn { border-radius: 8px; }
    </style>
</head>
<body>

<div class="container mt-5">
    <div class="container-card">
        <h2 class="text-center mb-4">Student Management System</h2>

        <div class="row g-3 mb-4">
            <div class="col-md-3">
                <input type="text" id="search" class="form-control" placeholder="🔍 Search students...">
            </div>
            <div class="col-md-3">
                <input type="text" id="name" class="form-control" placeholder="Full Name">
            </div>
            <div class="col-md-4">
                <input type="email" id="email" class="form-control" placeholder="Email Address">
            </div>
            <div class="col-md-2">
                <button id="addStudent" class="btn btn-primary w-100">Add Student</button>
            </div>
        </div>

        <table class="table table-hover">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Name</th>
```

```
                <th>Email</th>
                <th class="text-center">Actions</th>
            </tr>
        </thead>
        <tbody id="tableData">
            </tbody>
    </table>
  </div>
</div>

<div class="modal fade" id="editModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
        <div class="modal-header">
            <h5 class="modal-title">Edit Student Details</h5>
            <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
        </div>
        <div class="modal-body">
            <input type="hidden" id="editId">
            <div class="mb-3">
                <label class="form-label">Name</label>
                <input type="text" id="editName" class="form-control">
            </div>
            <div class="mb-3">
                <label class="form-label">Email</label>
                <input type="email" id="editEmail" class="form-control">
            </div>
        </div>
        <div class="modal-footer">
            <button id="updateStudent" class="btn btn-success">Save Changes</button>
        </div>
    </div>
  </div>
</div>

<script src="script.js"></script>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
</body>
```

```
</html>
```

script.js

JavaScript

```
$(document).ready(function() {
    loadData();

    // Load Data
    function loadData(search = "") {
        $.ajax({
            url: "fetch.php",
            type: "GET",
            data: { search: search },
            success: function(data) {
                $("#tableData").html(data);
            }
        });
    }

    // Insert Student
    $("#addStudent").click(function() {
        let name = $("#name").val();
        let email = $("#email").val();

        if (name == "" || email == "") {
            alert("Please fill all fields");
            return;
        }

        $.ajax({
            url: "insert.php",
            type: "POST",
            data: { name: name, email: email },
            success: function(response) {
                if(response == "1") {
                    loadData();
                    $("#name").val("");
```

```
            $("#email").val("");
        } else {
            alert("Error saving data.");
        }
    }
  });
});

// Delete Student
$(document).on("click", ".deleteBtn", function() {
  if (confirm("Are you sure you want to delete this student?")) {
    let id = $(this).data("id");
    $.ajax({
        url: "delete.php",
        type: "POST",
        data: { id: id },
        success: function() {
            loadData();
        }
    });
  }
});

// Open Edit Modal
$(document).on("click", ".editBtn", function() {
  $("#editId").val($(this).data("id"));
  $("#editName").val($(this).data("name"));
  $("#editEmail").val($(this).data("email"));
  $("#editModal").modal("show");
});

// Update Student
$("#updateStudent").click(function() {
  let id = $("#editId").val();
  let name = $("#editName").val();
  let email = $("#editEmail").val();

  $.ajax({
```

```
            url: "update.php",
            type: "POST",
            data: { id: id, name: name, email: email },
            success: function() {
                $("#editModal").modal("hide");
                loadData();
            }
        });
    });

    // Search Live
    $("#search").on("keyup", function() {
        loadData($(this).val());
    });
});
```

4. Backend Operations

fetch.php

PHP

```
<?php
include 'db.php';
$search = $_GET['search'] ?? '';
$searchTerm = "%$search%";

$stmt = $conn->prepare("SELECT * FROM students WHERE name LIKE ? OR email LIKE ? ORDER
BY id DESC");
$stmt->bind_param("ss", $searchTerm, $searchTerm);
$stmt->execute();
$result = $stmt->get_result();

if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "<tr>
                <td>{$row['id']}</td>
                <td>{$row['name']}</td>
                <td>{$row['email']}</td>
                <td class='text-center'>
```

```
                <button class='btn btn-sm btn-outline-success editBtn' data-id='{$row['id']}'
data-name='{$row['name']}' data-email='{$row['email']}'>Edit</button>
                <button class='btn btn-sm btn-outline-danger deleteBtn'
data-id='{$row['id']}'>Delete</button>
              </td>
            </tr>";
    }
} else {
    echo "<tr><td colspan='4' class='text-center text-muted'>No students found.</td></tr>";
}
?>
```

insert.php

PHP

```
<?php
include 'db.php';
if (isset($_POST['name']) && isset($_POST['email'])) {
    $stmt = $conn->prepare("INSERT INTO students (name, email) VALUES (?, ?)");
    $stmt->bind_param("ss", $_POST['name'], $_POST['email']);
    echo $stmt->execute() ? "1" : "0";
}
?>
```

update.php

PHP

```
<?php
include 'db.php';
if (isset($_POST['id'])) {
    $stmt = $conn->prepare("UPDATE students SET name=?, email=? WHERE id=?");
    $stmt->bind_param("ssi", $_POST['name'], $_POST['email'], $_POST['id']);
    echo $stmt->execute() ? "1" : "0";
}
?>
```

delete.php

PHP

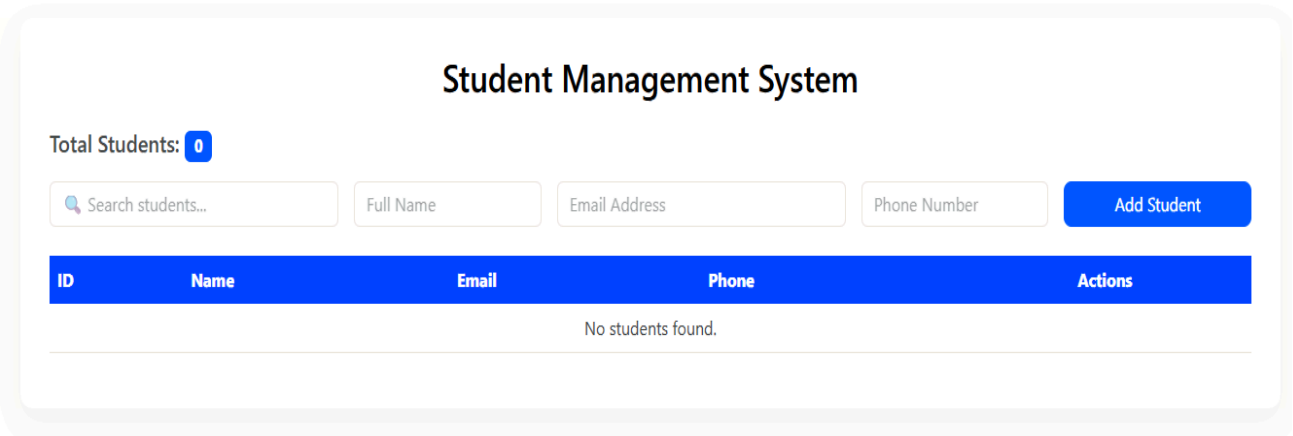
```
<?php
include 'db.php';
```

```
if (isset($_POST['id'])) {
    $stmt = $conn->prepare("DELETE FROM students WHERE id=?");
    $stmt->bind_param("i", $_POST['id']);
    echo $stmt->execute() ? "1" : "0";
}
?>
```

Student Task:

Task 1: Add a Phone Number Field

Modify the application to include a **phone number** field for each student.



Code Changed:

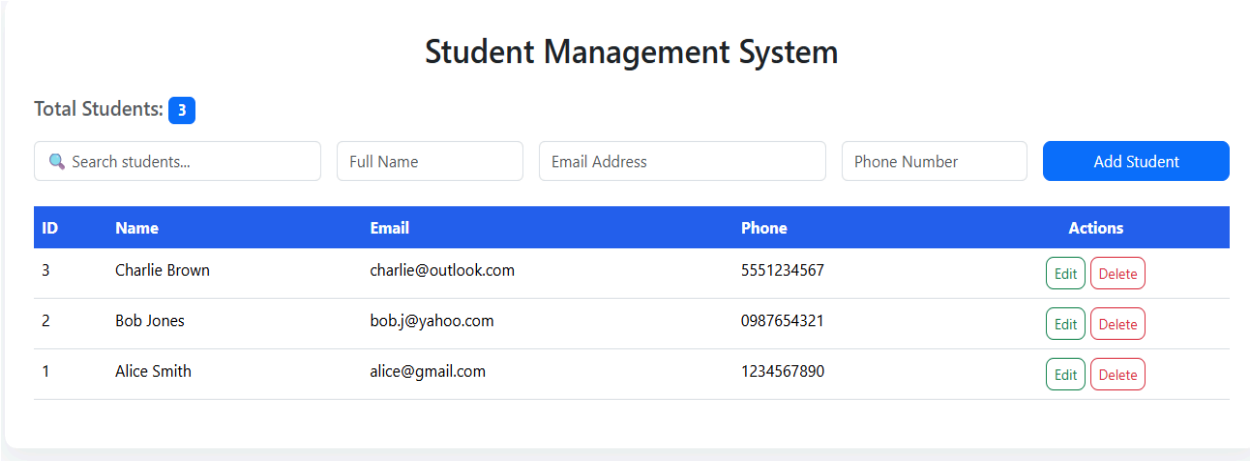
1. **Database updated to include the `phone` field (VARCHAR 20).**
2. **index.php: Inserted a `phone` input field in the Add and Edit forms.**

html

```html
<!-- HTML Form snippet -->
<div class="col-md-2">
   <input type="text" id="phone" class="form-control" placeholder="Phone Number">
</div>
```

Task 2: Display Total Student Count

Display the total number of students stored in the database above the table.



Code Changed:

1. **index.php: Added a badge element to display the total student count.**

html

```html
<h5 class="m-0 text-muted">Total Students: <span id="totalStudents" class="badge bg-primary">0</span></h5>
```

2. **fetch.php: Run a `COUNT(*)` query and returning the total alongside HTML via JSON.**
3. **script.js: Updates the counter when data loads.**

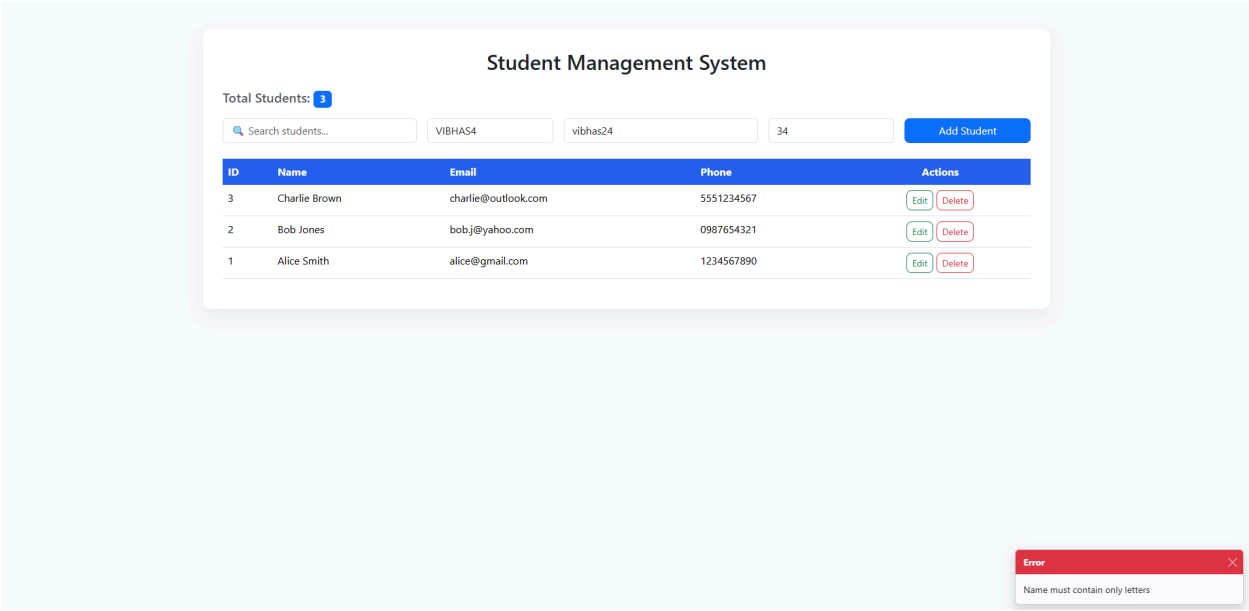
javascript

```javascript
if (response.total !== undefined) {
   $("#totalStudents").text(response.total);
}
```

Task 3: Add Form Validation

Add JavaScript validation to ensure:

- Name must contain only letters
- Email must follow valid format
- Fields cannot be empty



Code Changed:

1. **script.js: Added a robust JavaScript validation mechanism to ensure details are accurate before allowing any Database query.**

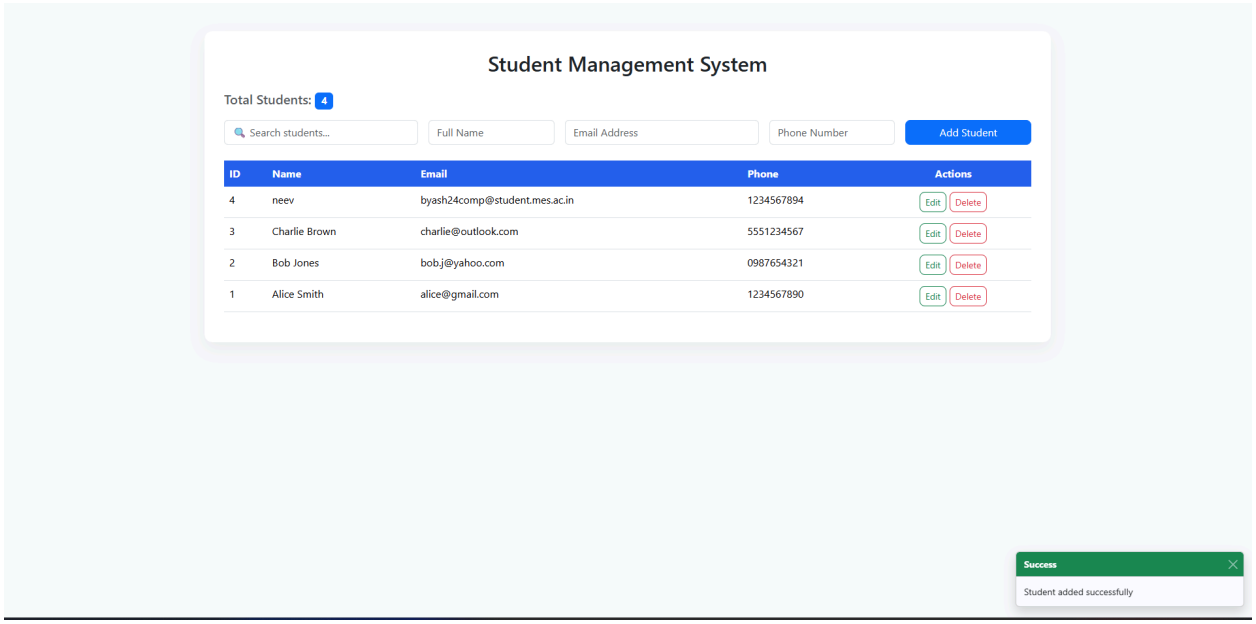
javascript

```javascript
function validateForm(name, email, phone) {
    if (name === "" || email === "" || phone === "") {
        showToast("Please fill all fields", "error");
        return false;
    }

    let nameRegex = /^[A-Za-z\s]+$/;
    if (!nameRegex.test(name)) {
        showToast("Name must contain only letters", "error");
        return false;
    }

    let emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
        showToast("Invalid email format", "error");
        return false;
    }
    return true;
}
```

Task 4: Add Toast Notifications

Replace alert messages with **Bootstrap toast notifications** when:

- Student is added
- Student is updated
- Student is deleted



Code Changed:

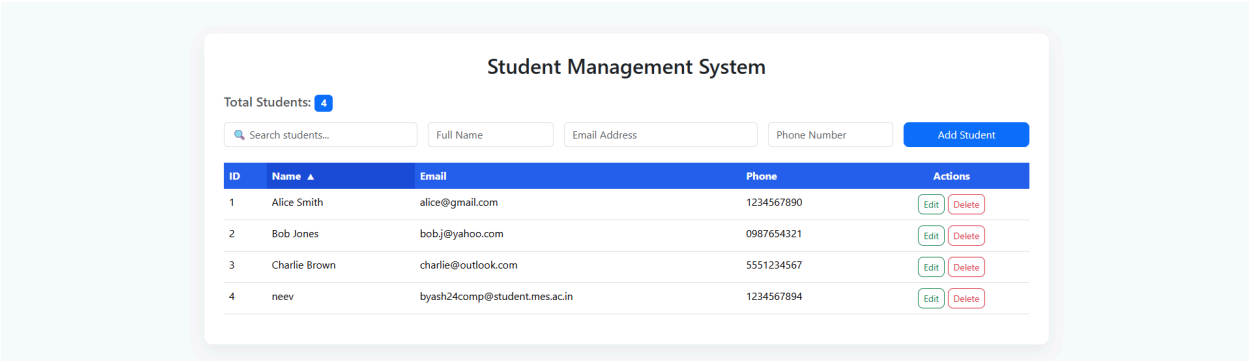
1. **index.php: Inserted Bootstrap Toast container code at the bottom of the page.**
2. **script.js: Created a custom**
3. **showToast() function to trigger whenever CRUD operations execute, bypassing the ugly default alerts.**

javascript

```javascript
function showToast(message, type = "success") {
   // Finds elements and binds them to the Bootstrap Toast class
   document.getElementById('toastBody').textContent = message;
   // Toggles between red or green background based on success state
   const toast = new bootstrap.Toast(document.getElementById('liveToast'));
   toast.show();
}
```

Task 5: Add Sorting Functionality

Modify the table so that clicking on **Name** or **Email** column headers sorts the records.



Code Changed:

1. **index.php: Set class `"sortable"` and designated SQL column targeting strings to Name and Email headers.**
2. **script.js: Intercepts column clicks, loops ascending or descending modes, and outputs a UI arrow (▲/▼).**

javascript

```javascript
$(".sortable").click(function() {
   let col = $(this).data("sort");
   if (currentSortColumn === col) {
       currentSortOrder = currentSortOrder === "ASC" ? "DESC" : "ASC";
   } else {
       currentSortColumn = col;
       currentSortOrder = "ASC";
   }
   // Automatically trigger Fetching
   loadData($("#search").val());
});
```

3. **fetch.php: Binds the sort to SQL string.**

php

```php
$stmt = $conn->prepare("SELECT * FROM students WHERE ... ORDER BY $sort_col
$sort_order");
```